

Constraint-Aware Pick-Throw-Place: Data-Efficient Model-Based Learning of Whole-Trajectory Feasible Release States for Low-Torque Manipulators

Anonymous submission

Abstract—Pick-throw-place (PTP) can extend a robot’s effective workspace for applications such as sorting and material handling: an object released with a controlled velocity follows a ballistic trajectory during free flight and can be deposited beyond the arm’s reach. Accurate throwing is difficult for manipulators with limited torque and velocity capabilities, and becomes harder still when aerodynamic drag on the object is not negligible. Recent model-based reinforcement learning (MBRL) approaches enable data-efficient throwing from few physical trials. However, these approaches typically rely on a predefined release configuration that may become infeasible when transferred to manipulators with lower torque and velocity limits. We address this limitation by introducing a constraint-aware release-state search that selects the release posture, direction, and achievable release speed according to arm-specific limits (joint position, joint velocity, and actuator torque). Candidate release states are further evaluated over the complete windup, throw, and post-release follow-through trajectories, rather than only at the release instant. The resulting release-state table is integrated with a sparse Gaussian-process dynamics model and particle-based policy optimization to learn target-dependent release speeds. Across four robotic manipulators in rigid-body simulation—KUKA iiwa7, Franka Panda, xArm6, and Kinova Gen3—the learning pipeline yields 1.4–2.7 cm mean landing error without arm-specific policy tuning, and the release-state search transfers to a 6-DOF UR7e without algorithmic change. A single target-conditioned policy transfers over a continuous range of target heights (0–0.45 m), giving 2.13 cm mean error at heights never seen during training. A drag sweep places the crossover at which learning starts to pay: below 1% drag the no-drag analytical baseline is the more accurate model, and at 19% it is 4.3–4.6 $\times$ worse. Finally, hardware experiments on the Kinova Gen3 execute the full stack with an object in the gripper and quantify the release-timing terms that govern transfer: a command lag that shortens the throw, a late object departure that lengthens it, and the instant at which they cancel.

Index Terms—Pick-throw-place, model-based reinforcement learning, Gaussian process regression, constraint-aware motion planning, and dynamic manipulation

I. INTRODUCTION

Pick-throw-place (PTP) provides an efficient alternative to conventional pick-and-place for applications such as sorting and material handling. By releasing an object with a controlled velocity and letting it follow a ballistic trajectory during free flight, a manipulator can deposit objects beyond its conventional end-effector workspace and reduce the transport motion required for each cycle. However, accurate throwing requires a fast and precisely timed release that remains within the manipulator’s kinematic and dynamic limits (joint position, joint velocity, and actuator torque). Small errors in release position, velocity, or timing can result

OFFLINE — arm-specific

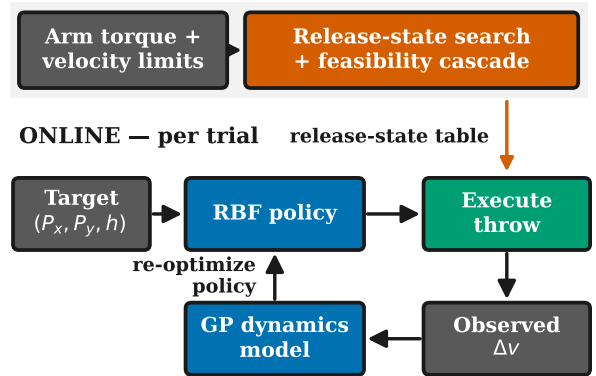


Fig. 1. Overview of the proposed constraint-aware model-based PTP framework. The offline stage is executed once per manipulator; the online stage repeats once per training trial.

in substantial landing errors, while limited actuator torque and joint velocity can prevent the arm from generating a useful release motion in the first place.

Recent model-based reinforcement learning (MBRL) methods have made robotic throwing substantially more data efficient. MC-PILOT [1], for example, combines sparse Gaussian-process (GP) dynamics learning with particle-based policy optimization to learn target-dependent release speeds from a small number of physical trials. Building on MC-PILCO [2] and earlier model-based tossing work [3], this formulation reduces the learning problem by prescribing a release configuration and analytically generating the joint motion that reaches it. This decomposition is effective on the 7-DOF Franka Emika Panda, but it assumes that the prescribed release configuration can be realized by the manipulator.

However, this assumption becomes restrictive when the same throwing strategy is transferred to a manipulator with lower actuator torque capacity. On the Kinova Gen3, whose wrist actuators provide substantially less torque than those of the Franka Emika Panda, directly adopting the fixed-release strategy does not produce a useful high-elevation throw. Under the predefined release posture, increasing the release elevation reduces the achievable range, causing distance-based optimization to favor a near-horizontal motion rather than a genuine throw. This observation indicates that the release configuration cannot be treated as a platform-

independent design constant; instead, it must be selected according to the kinematic and dynamic capabilities of the specific manipulator.

Analytical formulations provide fast and interpretable solutions by deriving release conditions from ballistic and manipulator models. Lynch and Mason [4] established throwing, catching, and batting as forms of dynamic manipulation that exploit gravity and momentum. More recent work has characterized reachable tossing workspaces and learned release-velocity mappings for bimanual systems [5]. While these approaches provide useful geometric and dynamical insight, their performance depends on model fidelity and on the feasibility of the resulting motion.

Learning-based methods reduce dependence on an exact analytical model. TossingBot [6] learns throwing directly from robot interaction using residual physics, while Monastirsky et al. [7] formulate throwing as sequence modeling to transfer simulation-trained behavior using limited physical data. Other approaches extend dynamic manipulation to bimanual handover [8], whole-body manipulation [9], and mobile-manipulator catching [10]. These methods demonstrate the effectiveness of learning for dynamic manipulation, but they do not directly address how to select a feasible release motion for a fixed-base, low-torque manipulator while retaining the data efficiency of model-based policy search.

A related line of work considers actuator and trajectory constraints explicitly. Manipulator trajectory optimization commonly incorporates joint-position, velocity, acceleration, and torque limits, which are particularly important for high-speed motions. Werner et al. [11] exploit passive joints of an underactuated, torque-limited material-handling machine to generate dynamic throws beyond its static reach, while Liu et al. [12] formulate planar throwing feasibility for online replanning under disturbance. Moura et al. [13] demonstrate constraint-aware trajectory optimization for dynamic manipulation. These methods motivate considering feasibility throughout the motion; however, they do not combine arm-specific release-state selection with sparse-GP model-based throwing on a fixed-base, torque-limited manipulator.

Motivated by these limitations, we propose a constraint-aware model-based framework for data-efficient PTP on manipulators with limited kinematic and dynamic capabilities. Our key contributions are summarized as follows. First, we propose a novel arm-specific release-state search that determines the release posture, direction, and achievable release speed from the manipulator’s own kinematic and dynamic limits, in place of a release configuration prescribed a priori. Second, we introduce a whole-trajectory feasibility cascade that evaluates the windup, release, and post-release follow-through motions rather than checking feasibility only at the release instant, enabling us to quantify its effect on the achievable throwing range. Third, we integrate the resulting release-state representation with sparse Gaussian-process dynamics learning and particle-based policy optimization to enable data-efficient target-dependent throwing, and investigate target-height generalization – whether one policy conditioned

on target height covers an interval of heights rather than a few discrete ones – and the operating regimes in which model-based learning improves over a no-drag analytical model. Finally, we demonstrate that the formulation is not tied to one arm, and we bring the same release solver to a physical Kinova Gen3, validating the execution components it requires. An overview of the proposed offline and online pipeline is shown in Fig. 1.

II. CONSTRAINT-AWARE MODEL-BASED PICK-THROW-PLACE

We formulate PTP as a model-based policy-search problem in which object dynamics are learned from a small number of throwing trials and a target-conditioned policy determines the release speed. Following MC-PILOT [1], we use sparse Gaussian-process (GP) dynamics and particle-based policy optimization. However, rather than fixing the release configuration a priori, we first determine a release state that is consistent with the kinematic and dynamic limits of the selected manipulator.

The resulting framework separates arm-specific feasibility computation from data-efficient model learning. An offline search identifies feasible release postures, directions, and release speeds and stores them in an azimuth-indexed release-state table. Online, the policy selects the release speed for the target, the corresponding release state is retrieved from the table, and observed throwing transitions are used to update the GP model. As shown in Fig. 1, the offline release-state search depends only on the manipulator’s torque and velocity limits and is therefore executed once per arm and workspace, whereas the policy, the executed throw, and the GP model form a loop that repeats once per training trial. The complete procedure is summarized in Algorithm 1.

A. Problem Formulation and Learning Framework

The object dynamics are represented by three independent sparse GPs, one for each Cartesian velocity component. Each GP models the velocity increment

$$(\mathbf{p}, \mathbf{v}) \rightarrow \Delta \mathbf{v}, \quad (1)$$

where $\mathbf{p} \in \mathbb{R}^3$ and $\mathbf{v} \in \mathbb{R}^3$ denote the object position and velocity. The models are learned from transitions collected during throwing trials and are subsequently used to propagate particles during policy optimization. Each GP is sparse: rather than retaining every collected transition, it is summarized by a fixed set of M inducing points and trained on mini-batches of N_b transitions, which bounds both memory and per-iteration cost as trials accumulate.

The policy is represented by a radial basis function (RBF) model that maps the target state to a release speed. For the standard formulation, the target position is used as input; for target-height generalization, the target height is included as an additional input. The unconstrained policy output x is mapped to the feasible release-speed interval $[0, u_M]$ as

$$u = \frac{u_M}{2} (\tanh(x) + 1), \quad (2)$$

Algorithm 1 Constraint-Aware Model-Based Pick-Throw-Place

Offline release-state search (*once per arm/workspace*)

- 1: **Input:** URDF; limits $\dot{q}^{\max}, \tau_{\max}$; target range
- 2: $\mathcal{C} \leftarrow \emptyset$
- 3: **for** each posture q and static offset on the grid **do**
- 4: **if** EE height or static-gravity torque infeasible **then**
- 5: **continue**
- 6: **end if**
- 7: **for** each direction $\hat{d}$ (elev. 5° – 45° , both signs) **do**
- 8: $s^* \leftarrow$ inner LP (3)
- 9: **if** candidate passes feasibility cascade **then**
- 10: $R \leftarrow$ landing distance from drag-augmented integration
- 11: $\mathcal{C} \leftarrow \mathcal{C} \cup \{(q, \hat{d}, s^*, R)\}$
- 12: **end if**
- 13: **end for**
- 14: **end for**
- 15: $(q, \hat{d})^{\text{rel}} \leftarrow \arg \max_{\mathcal{C}} R$; propagate over azimuth $\rightarrow \mathcal{T}$

Online model-based learning

- 16: collect exploration throws; fit sparse GP dynamics $\mathcal{G}$
- 17: **for** trial = 1, ..., N **do**
- 18: optimize RBF policy π against $\mathcal{G}$ using particle rollout
- 19: execute $\pi \rightarrow s$; retrieve release state from $\mathcal{T}$
- 20: execute computed-torque trajectory (5)
- 21: measure release velocity and landing outcome; update $\mathcal{G}$
- 22: **end for**

where u_M is the maximum release speed associated with the selected release state.

During policy optimization, particles are propagated through the learned GP dynamics and the policy parameters are optimized to minimize the expected landing distance to the target. Since optimization is performed against the learned model, the policy can be re-optimized between physical trials without additional robot interaction. The release-state search therefore provides the constraint-aware action space, while model-based learning determines the target-dependent release speed within that space.

B. Release-State Search

The release state is defined by the joint configuration and velocity $q, \dot{q} \in \mathbb{R}^n$, with $\tau_{\max}$ and $\dot{q}^{\max}$ denoting the corresponding manufacturer actuator limits ($n = 7$ for the manipulators considered here). Rather than fixing q_{rel} , we search over candidate postures and release directions and determine the maximum feasible release speed for each candidate. This search separates geometric selection from the velocity optimization required to realize a specified release direction.

For the 7-DOF Kinova Gen3, the three pitch joints provide the dominant contribution to the end-effector velocity in the selected swing plane. The four roll/twist joints are therefore

assigned zero release velocity, yielding the linear program

$$\begin{aligned} \max_{\dot{q}, s} \quad & s \\ \text{s.t.} \quad & J(q)\dot{q} = s \hat{d}, \\ & |\dot{q}_i| \leq \dot{q}_i^{\max}, \quad \forall i, \\ & \dot{q}_i = 0, \quad i \in \{0, 2, 4, 6\}. \end{aligned} \quad (3)$$

Here, s is the release speed, $\hat{d}$ is the desired unit release direction, and $J(q)$ is the manipulator Jacobian obtained by forward kinematics at the release configuration. The problem is linear in the decision variables $(\dot{q}, s)$, so its global optimum is obtained directly by linear programming and no initial guess or local search is required.

The static angles of the roll/twist joints remain free search variables. Their release velocities are set to zero to avoid introducing motion components that are poorly aligned with the desired throwing direction, while their static offsets also prevent geometric degeneracy of the pitch-joint swing mechanism.

Candidate postures are generated by grid-searching the relevant joint angles and static offsets. Each posture is first checked by forward kinematics for the required end-effector height, and by inverse dynamics for the static gravity-induced torque it demands. For feasible postures, release directions are sampled within the corresponding swing plane using elevation angles from 5° to 45° in 5° increments and both elevation signs. The inner optimization in Eq. (3) then provides the maximum release speed for each posture–direction pair.

The resulting candidates are evaluated using the complete trajectory feasibility cascade described in Sec. II-C. Among the candidates that satisfy all constraints, the release state maximizing the predicted landing distance is selected. The landing distance is obtained by forward integration of the drag-augmented ballistic dynamics using the candidate’s actual release position and velocity.

Once the release state has been verified, it is propagated over the required base-azimuth range by rotating the base joint. Since gravity is invariant to rotation about the vertical axis, the corresponding Jacobian and feasibility properties rotate consistently with the base angle. We therefore construct an azimuth-indexed release-state table rather than repeating the full search for each target bearing. For the Gen3, the verified state is propagated from -33° to $+33^\circ$ in 3° increments, with a turret-aiming correction accounting for the offset between the tabulated release geometry and the target bearing. A single release-solving procedure maps a commanded release speed and target to the corresponding release state, and the same procedure is invoked during training and during hardware execution, so that the motion the policy is optimized against is the motion the manipulator performs.

C. Whole-Trajectory Feasibility

A release state that satisfies constraints at the release instant may still be unexecutable during windup or recovery. We therefore evaluate the complete throwing motion using a

sequential feasibility cascade. A candidate is rejected at the first stage that violates the specified constraint.

First, the windup configuration is computed from the release state as

$$q_{\text{windup}} = q_{\text{rel}} - \dot{q}_{\text{rel}} \frac{t_{\text{throw}}}{2}, \quad (4)$$

and must satisfy all joint-position limits. The neutral-to-windup trajectory is then sampled at five intermediate configurations, where gravity-induced inverse dynamics are evaluated and constrained to remain below 90% of the corresponding manufacturer torque limit.

Next, the cubic velocity profile used by the runtime controller to move from windup to release is evaluated at 40 points. At each point, the required torque includes the manipulator inverse dynamics and the Jacobian-transpose feedforward term associated with the weight of the gripped object. The torque must remain below 75% of the corresponding actuator limit.

Finally, the post-release recovery trajectory is evaluated for eight candidate recovery durations between 0.6 and 4.0 s, using 30 samples per duration. Both joint torque and velocity limits are checked, and a candidate is considered feasible if at least one recovery duration satisfies the specified margins.

This procedure captures constraint violations that cannot be detected from the release state alone, including excessive windup torque, release-ramp torque, and recovery demands following a high-speed throw.

The resulting motion is shown in Fig. 2 for the Kinova Gen3. The arm first travels backwards and upwards from its neutral posture into the windup configuration (Fig. 2a), then sweeps forward through the release state (Fig. 2b), where the object separates from the gripper and follows a ballistic path (Fig. 2c) to the target (Fig. 2d). The post-release recovery, executed while the object is in flight, carries the arm to rest and is the phase that most often exceeds the Gen3's wrist limits, because it must dissipate the same joint velocities that the throw was optimized to maximize. The release velocity in this sequence is generated by the simulated arm under computed torque and measured after the grip constraint is removed, rather than assigned to the object.

D. Dynamics-Aware Throw Execution

The selected release state is evaluated using the coupled manipulator and object dynamics. The manipulator is controlled using computed torque,

$$\tau = M(q)\ddot{q}_{\text{cmd}} + C(q, \dot{q}) + G(q) + J(q)^\top f_{\text{ball}}, \quad (5)$$

where M , C , and G denote the mass, Coriolis/centrifugal, and gravity terms, respectively. The payload compensation term is

$$f_{\text{ball}} = m_{\text{ball}}[0, 0, g]^\top. \quad (6)$$

The object is modeled as a separate rigid body attached through a removable grip constraint. The attachment point is offset from the wrist flange by the tool-center-point distance $r_{\text{offset}} = 0.12$ m. This offset is included consistently in the release-state optimization and rigid-body simulation. In

particular, the release velocity contains the rotational contribution

$$\mathbf{v}_{\text{offset}} = \boldsymbol{\omega} \times \mathbf{r}_{\text{offset}}, \quad (7)$$

which would be omitted if the object were attached directly to the wrist flange. The same attachment point is therefore used for release-state calculation and simulation.

The release velocity is measured from the physics engine after the grip constraint is removed rather than imposed directly. This ensures that the reported release state corresponds to the velocity generated by the simulated arm-object system.

E. Target-Height Generalization

The target-conditioned formulation is extended to targets at different heights by parameterizing both the trajectory termination plane and particle propagation with the target height h . The policy input consequently becomes (P_x, P_y, h) .

We consider three settings: training at a single fixed height, re-optimizing the policy for discrete target heights while reusing the learned GP, and training a single policy over a continuous range of target heights. The latter tests whether the learned policy can interpolate to target heights not observed during training without requiring a separate policy-optimization run for each height.

III. RESULTS AND DISCUSSION

A. Simulation Environment and Evaluation

All simulation experiments are conducted in PyBullet [14] using rigid-body dynamics and manufacturer-specified joint torque and velocity limits. We evaluate KUKA iiwa7, Franka Panda, xArm6, and Kinova Gen3 under the same model-based learning procedure, three seeds each, with only the arm model and its velocity limits varied; these four runs use each arm's prescribed release configuration, so they isolate the learning pipeline's arm-independence. The release-state search itself is run on the Kinova Gen3 and, in Sec. III-I, the Franka Panda and UR7e. The Gen3 results are torque-controlled with the release velocity measured from the physics engine; the KUKA and Panda runs are position-controlled, and the xArm6 assigns the release velocity directly, so only the Gen3 numbers are physically realized throws. Gen3 targets are sampled in flight space relative to the actual release point. Final performance is evaluated on previously unseen targets and random seeds using complete rigid-body rollouts. Landing error is the Euclidean distance between the simulated landing position and the target. A throw counts as successful when this error falls below a stated threshold; we report two, a 10 cm threshold matching the bin radius used in [1] and a tighter 5 cm threshold. Seed counts are stated with each result. Where conditions are compared against one another, they are evaluated on a single shared set of previously unseen targets, so that only the condition varies; comparisons across different manipulators are necessarily an exception, since each arm has its own reachable target band.

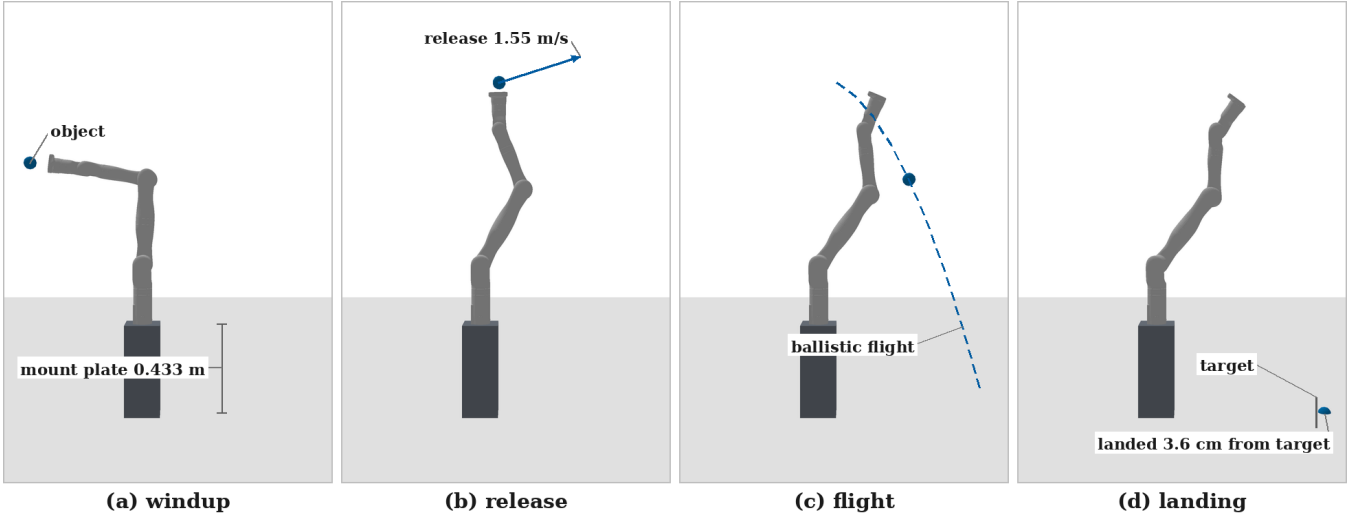


Fig. 2. Simulated Kinova Gen3 throw: (a) windup, (b) release, (c) flight during post-release recovery, and (d) landing. Frames are from a single rigid-body rollout of the learned policy under computed-torque control; the dashed curve in (c) is the object’s recorded trajectory.

B. Hardware Environment

Hardware validation uses a 7-DOF Kinova Gen3 mounted approximately 0.433 m above the floor. The planned trajectory is executed as a joint-velocity stream issued through the manufacturer’s high-level real-time control interface, whose command rate is approximately 40 Hz. An Intel RealSense D435i provides visual sensing; the camera-to-base transform is recovered by observing a planar fiducial target of known dimensions and composing the resulting camera-to-target pose with the forward kinematics of the arm. Landing position is measured by triangulating the object in flight from the rectified infrared stereo pair and solving a ballistic fit for the descending crossing of the floor plane, rather than from block-matching depth, which is too noisy at this range for a small textureless sphere. The experiments are staged, each stage validated before the next is enabled. They cover state feedback, gripper actuation, velocity streaming, and object-loaded throws; the object’s departure instant and its landing point remain unmeasured, so physical throwing accuracy is not evaluated.

C. Training and Implementation Details

The RBF policy length scale is initialized as

$$\ell_0 = 0.15(l_M - l_m), \quad (8)$$

where $[l_m, l_M]$ is the target-distance range. The sparse GP uses $M = 400$ inducing points and mini-batches of $N_b = 250$. Simulation runs at 50 Hz and the Gen3 velocity interface at approximately 40 Hz. All remaining policy and dynamics parameters follow Sec. II.

D. Optimization Reliability

We first evaluate the reliability of model-based policy optimization across random seeds. A single baseline run at a fixed seed achieves 5/5 successful throws in five trials, at 1.81 cm mean landing error, which taken alone would suggest the configuration is reliable. Repeating that same

configuration over five seeds instead yields hit rates of 60%, 20%, 80%, 10%, and 10%, indicating substantial sensitivity to initialization and exploration.

Two interacting factors account for this sensitivity. Unstratified exploration can leave the GP poorly informed over parts of the release-speed range, while an RBF lengthscale that is not scaled to the target domain can saturate the policy response. Fig. 3 separates the two: stratified exploration alone recovers three of the five seeds, and only when it is combined with target-range-scaled lengthscale initialization do all five converge. Neither modification is therefore sufficient on its own.

With both modifications applied, the five seeds are evaluated on 50 previously unseen targets each, yielding 250 throws. Every throw falls within both the < 10 cm and the < 5 cm threshold, giving a 100% hit rate at both, with a mean landing error of 1.86 cm and a 95th-percentile error of 3.17 cm. Fig. 4 shows that this is consistent across seeds rather than an average over dissimilar runs: per-seed mean error spans only 1.77–1.96 cm, and the worst single throw of all 250 is 3.81 cm, still inside the tighter threshold.

E. Cross-Manipulator Evaluation

The model-based learning pipeline is evaluated on KUKA iiwa7, Franka Panda, xArm6, and Kinova Gen3 without arm-specific policy tuning beyond the corresponding velocity limits. Mean landing error ranges from 1.4 to 2.7 cm across the four manipulators, a spread comparable to the seed-to-seed variation observed on a single arm in Sec. III-D. The learning pipeline is therefore not tied to the Gen3 geometry. Transfer of the release-state search itself is a separate question, addressed on the Franka Panda and the 6-DOF UR7e in Sec. III-I.

F. Aerodynamic-Drag Robustness

We vary payload drag from less than 1% to approximately 19% of gravity and compare the learned model with the

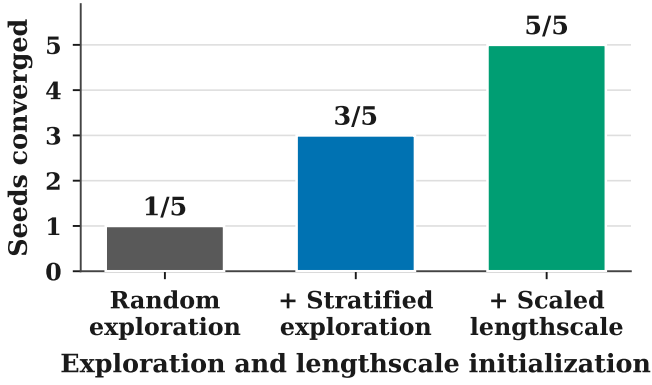


Fig. 3. Seeds converged, out of five, under random exploration and under each reliability modification in turn.

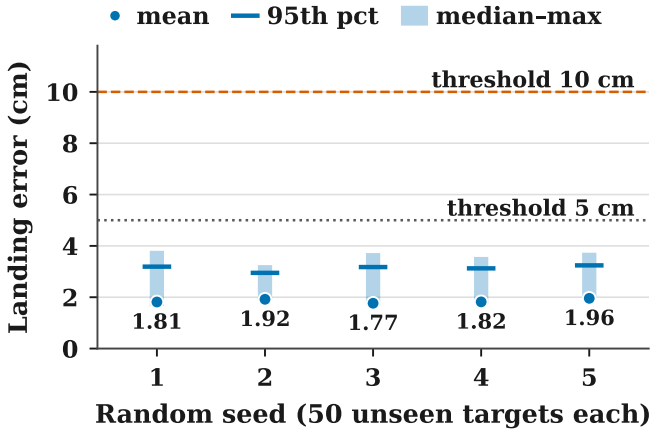


Fig. 4. Final-policy landing error per random seed, on 50 previously unseen targets each.

analytical no-drag baseline. Three seeds per condition. Below 1% drag the closed-form baseline is not merely competitive but more accurate than the learned model (1.65 vs 1.86 cm on the KUKA, 1.43 vs 1.94 cm on the Panda), since there is no unmodeled effect to learn and learning still pays a variance cost. At 19% drag the ordering reverses by $4.3\text{--}4.6\times$ (5.34 vs 1.26 cm and 3.79 vs 0.82 cm).

We further vary payload mass from 30 to 150 g and diameter from 2 to 4.5 cm without retraining. All twelve mass–diameter combinations remain within 1.38–1.46 cm mean landing error. The analytical model therefore remains competitive when aerodynamic effects are small, while the learned dynamics provide increasing benefit as the unmodeled drag becomes significant.

As an additional ablation, subtracting gravity as a GP mean function does not improve accuracy: the mean error increases from 0.62 to 0.85 cm for the Panda and from 1.69 to 1.77 cm for the KUKA. This suggests that imposing gravity as the dominant structured component is not beneficial for the residual dynamics in the tested regime.

G. Target-Height Generalization

A single KUKA iiwa policy trained over a continuous target-height range is evaluated on 100 fresh targets at

heights not observed during training. It records 100% successful throws, a mean landing error of 2.13 cm, and a worst-case error of 4.92 cm. Restricted to the three heights used in the adaptation study below, the same policy gives 2.22, 2.03, and 2.12 cm mean error at $h = 0.10, 0.20$, and 0.30 m (targets within ± 2.5 cm of each height; $n = 4, 18$, and 13), showing no systematic trend with height. Fig. 5 plots this error against target height: the local mean stays between 1.7 and 3.0 cm across the entire 0–45 cm range, and in particular does not grow towards the boundaries of the trained interval, so the single conditioned policy interpolates over height rather than specializing to the centre of the range.

The preceding result uses a single policy conditioned on height and trained over the whole interval. We next ask what happens when no such policy is available, and evaluate zero-new-trial adaptation on the Gen3 overhead configuration by reusing the trained GP and re-optimizing only the policy for the same three target heights $h = 0.10, 0.20$, and 0.30 m. The two studies use different arms, so they bound different platforms rather than competing with each other directly. Because each re-optimized policy selects its own reachable target band, and those bands differ by more than an order of magnitude in width, all four conditions are evaluated on a single fixed set of 30 previously unseen targets drawn from the intersection of the four bands, so that target height is the only quantity that varies. On that shared set (Fig. 6), mean landing error grows monotonically from 1.52 cm at ground level to 1.82, 2.24, and 2.40 cm at $h = 0.10, 0.20$, and 0.30 m, with every throw inside the 10 cm threshold and a worst case of 5.18 cm. A complete nine-dimensional retraining baseline is not included in this comparison: the only such checkpoint on disk predates the gripper tool-center-point correction of Sec. II-D and was trained from a release point 0.39 m lower, so placing it alongside these numbers would conflate that correction with the retraining-versus-adaptation question this experiment is designed to isolate. Accuracy therefore degrades gradually and predictably with height, while adaptation costs no additional robot trials, so target-height changes can be accommodated without relearning the GP dynamics.

H. Low-Torque Throwing

We next evaluate the release-state search on the low-torque Kinova Gen3. Under a fixed low-release posture, increasing elevation reduces the achievable range, causing distance optimization to favor a near-horizontal motion. In contrast, searching over the release state under the complete feasibility cascade identifies an overhead throwing configuration.

The selected configuration reaches a release height of approximately 1.1 m. When windup, release, and post-release recovery constraints are enforced, the maximum simulated safe-throw distance is 1.03 m, against the 0.75 m at which this arm can place an object on the same floor – it is mounted 0.433 m up, so reaching down to floor level costs horizontal extent. Throwing therefore extends this arm’s floor-level placement radius by $1.4\times$. Over 30 previously unseen targets the mean landing error is 1.52 cm, the worst case 3.70 cm,

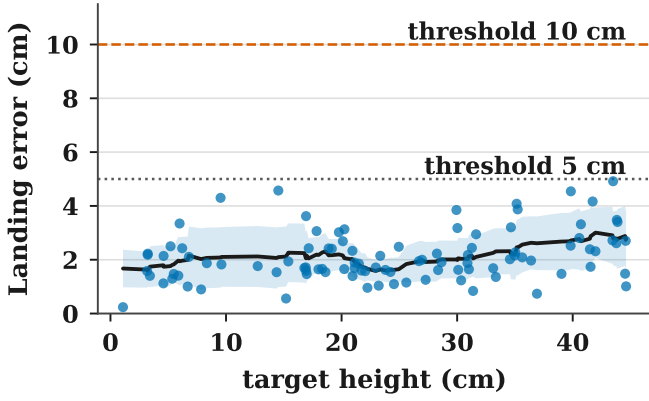


Fig. 5. Landing error against target height for the KUKA iiwa, on 100 targets at heights not seen during training. The line is a rolling mean, the band its standard deviation.

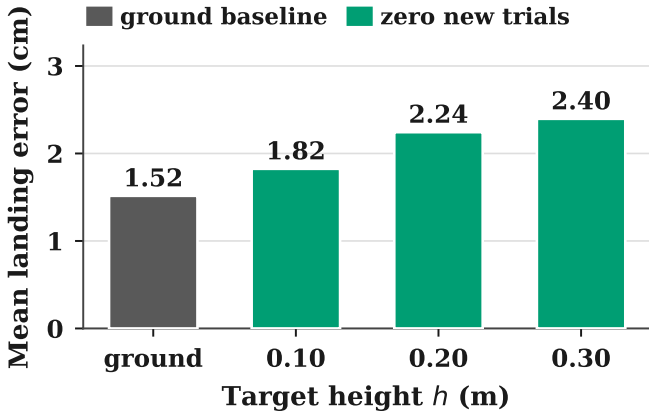


Fig. 6. Zero-new-trial target-height adaptation on the Kinova Gen3, all conditions evaluated on the same fixed set of 30 previously unseen targets.

and every throw falls within both thresholds. Release speed varies only from 1.35 to 1.47 m/s across the target-distance range, well inside the arm’s 2.07 m/s kinematic ceiling: the trained target band, not the actuators, bounds the deployed policy.

I. Whole-Trajectory Feasibility

We quantify the importance of whole-trajectory feasibility by applying the same release-pose and direction enumeration to the Gen3 and Franka Panda. Candidates that satisfy the release-instant criterion are subsequently evaluated using the complete feasibility cascade. The resulting counts are summarized in Table I.

For the Gen3, 391,238 of 475,123 release-instant-feasible candidates remain feasible after the complete cascade. Every rejection is attributed to the post-release follow-through: at this arm’s real torque limits, no candidate that reaches the release state fails on the windup path or the throw ramp. The rejected 17.7% is therefore a recovery constraint rather than a torque shortfall, and the sweep below shows that no additional actuator capacity removes it. The best achievable range is unchanged at 1.03 m, because the range-optimal candidate is itself whole-trajectory feasible: the cascade

TABLE I

RELEASE-INSTANT-ONLY VERSUS WHOLE-TRAJECTORY FEASIBILITY ACROSS THREE MANIPULATORS ON THE SAME POSTURE AND DIRECTION GRID. BODYLESS FRAMES IN EACH MANIPULATOR DESCRIPTION ARE GIVEN ZERO MASS; LOADING THE GEN3 DESCRIPTION UNMODIFIED INSTEAD ASSIGNS THEM THE SIMULATOR’S 1 KG DEFAULT, ADDING 3.00 KG AT THE WRIST AND REDUCING ITS SURVIVORS TO 74,230 (16.6%).

	Gen3	Panda	UR7e
Release-instant-feasible	475,123	152,844	541,590
Also full-traj.-feasible	391,238 (82.3%)	152,844 (100%)	541,569 (100.0%)
Rej. windup-path	0	0	0
Rej. throw-ramp	0	0	0
Rej. follow-through	83,885 (17.7%)	0	21
Best range, instant-only	1.03 m	1.65 m	2.55 m
Best range, full check	1.03 m	1.65 m	2.55 m

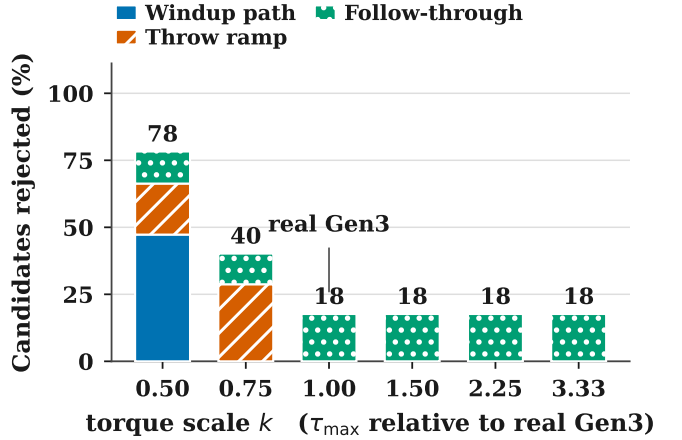


Fig. 7. Share of release-instant-feasible candidates rejected by each stage of the cascade, against actuator torque capacity, for Gen3-identical kinematics.

determines *which* release state is selected and whether the selected motion is executable, not how far the arm can throw.

For the Panda and the UR7e essentially every release-instant-feasible candidate also satisfies the cascade (100% and 99.996%), and neither maximum range moves. The cascade is thus cheap insurance on an arm with actuator headroom and decisive on one without, which is the argument for applying it unconditionally. Because the Gen3 and Panda differ in both kinematics and torque limits, their direct comparison does not isolate the effect of actuator torque. We therefore repeat the analysis using Gen3-identical kinematics and control gains while scaling only the torque limits by a factor k (Fig. 7, Table II).

Fig. 7 decomposes this rejection by cascade stage, and the three stages bind in sequence rather than together. At $k = 0.50$ the windup path dominates (47.3% of the pool), with throw ramp and follow-through at 19.0% and 12.0%: the arm struggles to reach a throwing posture at all. By $k = 0.75$ the windup path clears and the throw ramp binds at 28.7%. From $k = 1.00$ both clear and rejection saturates at

TABLE II

TORQUE-CAPACITY SWEEP USING GEN3-IDENTICAL KINEMATICS.
REJECTION RATES ARE COMPUTED RELATIVE TO THE
TORQUE-DEPENDENT RELEASE-INSTANT-FEASIBLE POOL.

k	wrist $\tau_{\max}$ (Nm)	instant- feasible	full-traj- feasible	rej. (%)
0.50	4.5	444,713	96,584	78.3
0.75	6.75	475,123	284,156	40.2
1.00 [†]	9.0	475,123	391,238	17.7
1.50	13.5	475,123	391,238	17.7
2.25	20.25	475,123	391,238	17.7
3.33 [‡]	29.97	475,123	391,238	17.7

[†]reproduces the real Gen3 torque limits. [‡]approximates the Panda torque ratio only and does not reproduce its kinematics.

17.7%, entirely follow-through. Saturation therefore begins exactly at the real Gen3’s own torque limits: this arm is not torque-starved for the throw itself, its binding constraint is already the recovery motion, which is kinematic in origin and unaffected by additional torque.

J. Computational Efficiency

A complete Gen3 overhead-throw training run with 10 trials, including five exploration trials and $N_{\text{opt}} = 1500$ policy-optimization steps per trial, requires 17 min 44 s on a single CPU core with 1.11 GB peak memory. The one-time release-state search enumerates 3.2×10^4 release postures against six candidate directions each and requires 28 min 27 s on the same machine, at a single azimuth that is then propagated by base rotation. These costs are incurred once per arm and workspace geometry, after which the release-state table is reused across training seeds and evaluation targets.

K. Hardware Validation

The physical Kinova Gen3 is used to validate the unloaded execution pipeline. Table III records how far that validation has progressed: each row is one capability the execution stack requires, and the pipeline is enabled one row at a time, so the two rows still marked pending are precisely what separates the present hardware study from a measured physical throw. State read-back, gripper actuation, full-speed joint-velocity streaming, and object-loaded throws have all been executed on the physical arm; what is not yet measured is when the object leaves the hand and where it lands.

Joint angles are handled using the $[0^\circ, 360^\circ)$ convention reported by the arm. Across $n = 9$ full-speed throws the arm lags the commanded profile by about one control interval: at the planned release instant the measured tool-centre speed is 0.948 ± 0.005 of commanded, the direction $3.82 \pm 0.47^\circ$ steeper, and joint drift 0.037 ± 0.004 rad, which propagate ballistically to 10.2 ± 0.9 cm short of plan. Gripper-open onset is 67.9 ± 6.7 ms ($n = 15$, empty) and 66.9 ± 15.2 ms ($n = 3$, on a ball) with the stream idle, and the executor leads the open command by that latency. In-throw recordings show the lead is ineffective: while joint velocities stream the

TABLE III

WHICH STAGES OF THE EXECUTION PIPELINE HAVE RUN ON THE
PHYSICAL KINOVA GEN3. WHAT SEPARATES THIS STUDY FROM A
PHYSICAL ACCURACY RESULT IS MEASUREMENT, NOT EXECUTION.

Execution stage	Executed on the arm
Read joint and gripper state	yes
Command the gripper	yes
Stream joint velocities at full speed	yes
Keep open-loop drift within budget	yes
Throw with the object in the gripper	yes
Compensate gripper latency at release	attempted, ineffective
Measure the object’s departure instant	not yet
Measure the physical landing point	not yet

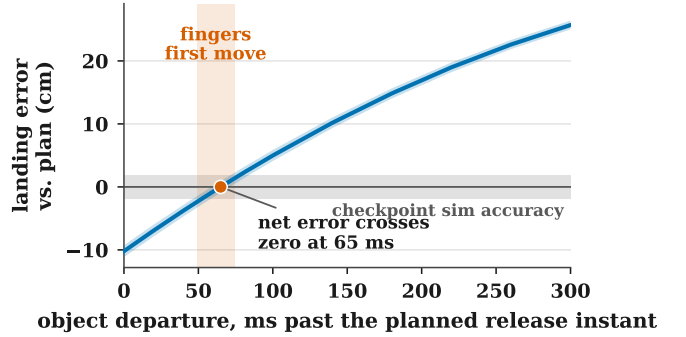


Fig. 8. Net landing error against the instant the object leaves the hand, from $n = 9$ recorded full-speed throws (band: ± 1 s.d.). The command lag makes the throw short and a late release makes it long, so the two cancel near 65 ms; the shaded column is where the fingers were observed to first move.

fingers do not move, and across $n = 4$ instrumented throws they first moved 49–74 ms *after* the planned release instant. The two dominant terms then oppose – during the stream pause the arm coasts at its held release velocity, so a late departure flattens the release and lengthens the throw while the command lag shortens it. Fig. 8 plots the net error against departure instant: -10.2 cm at onset, crossing zero at 65 ms. Neither can be corrected alone without risking a larger error of the opposite sign, and the 25 ms command interval sets a 3.6 cm floor beneath both.

These measurements establish the operation of the hardware execution stack at the full planned release speed. This exceeds the manufacturer’s documented 0.500 m/s Cartesian speed ceiling, but that ceiling applies to a Cartesian control mode distinct from the joint-space velocity streaming used here, as confirmed both by the arm’s reported mode limits and by a full-speed empty-gripper rehearsal completing without fault. What remains is measurement, not execution: the object’s departure instant and its landing point are both unresolved, so no physical landing accuracy is claimed.

L. Discussion

The results support three main conclusions. First, on an arm whose actuators cannot realize a prescribed release configuration, searching the release state is what makes the throw exist at all: the Gen3’s prescribed posture drives

distance optimization toward a near-horizontal push, and the search recovers a genuine overhead throw. Second, whole-trajectory feasibility matters most near actuator limits, but not for the reason a release-instant analysis suggests: at the Gen3's real torque limits neither the windup path nor the throw ramp rejects anything, and the only binding stage is the post-release recovery, which the torque sweep shows to be kinematic in origin and unaffected by added actuator capacity. Third, model-based learning becomes increasingly beneficial as the analytical no-drag model departs from the observed dynamics, while target-height conditioning enables adaptation without relearning the GP dynamics.

The results also clarify the role of the proposed framework. It does not remove the physical limits of the manipulator; rather, it identifies and exploits the subset of release states for which the complete throwing motion remains executable. The achievable throwing range is therefore jointly determined by kinematic reach, actuator torque, joint velocity, and post-release recovery constraints.

Five limitations bound these conclusions. The accuracy results are established in rigid-body simulation; the physical arm executes the full stack including object-loaded throws, but the object's departure instant and its landing point are both unmeasured, so no physical accuracy is claimed. The feasibility counts depend on whether bodyless frames in the manipulator description are given zero mass, which changes them fivefold on the Gen3 and not at all on the Panda. The release-state search is a discrete enumeration and therefore incurs a one-time computational cost that depends on grid resolution. Object and multi-manipulator generalization are currently evaluated in simulation, and the shared computed-torque gains have not been independently validated for each arm. In addition, the Gen3 implementation uses fixed lead-time compensation for measured gripper delay rather than learning a release-delay distribution. Finally, the current hardware study does not yet establish how the measured timing and drift effects combine during a physical loaded throw.

IV. CONCLUSION

We presented a constraint-aware model-based framework for robotic PTP in which the release state is selected according to the manipulator's kinematic and dynamic capabilities rather than fixed a priori. The framework combines arm-specific release-state search and whole-trajectory feasibility checking with Gaussian-process dynamics learning and particle-based policy optimization, enabling target-dependent throwing within the feasible operating envelope.

Across four simulated manipulators the learning pipeline yields 1.4–2.7 cm mean landing error without arm-specific policy tuning, and the release-state search transfers to a 6-DOF UR7e without algorithmic change. On the low-torque Kinova Gen3 it identifies a 1.03 m safe-throw boundary against the 0.75 m at which the same arm can place an object on that floor, at 1.52 cm mean landing error on previously unseen targets. The additional drag and target-height experiments show that learned dynamics become increasingly

beneficial when analytical dynamics are incomplete, while target-conditioned policies can adapt to unseen target heights without relearning the GP dynamics.

Physical Gen3 experiments exercise the execution stack end to end with an object in the gripper and quantify the release-timing terms that govern transfer. Measuring the object's departure instant and its landing point remains to be completed, together with broader real-world validation across manipulators and object conditions. Future work will therefore focus on physical throwing evaluation, improving the efficiency of the release-state search, and extending the framework to broader robot and object configurations. Thus, the present work establishes the simulation-side effectiveness and hardware execution readiness of the proposed constraint-aware formulation, with full real-world throwing evaluation left as the next step.

REFERENCES

- [1] N. Turcato, G. Giacomuzzo, M. Terrea, D. Allegro, R. Carli, and A. Dalla Libera, "Data efficient robotic object throwing with model-based reinforcement learning," *arXiv preprint arXiv:2502.05595*, 2025.
- [2] F. Amadio, A. Dalla Libera, D. Nikovski, R. Carli, and D. Romeres, "Model-based policy search via monte carlo gradient estimation with real system applications," *IEEE Transactions on Robotics*, vol. 38, no. 6, pp. 3879–3898, 2022.
- [3] N. Turcato, A. Dalla Libera, G. Giacomuzzo, and R. Carli, "Teaching a robot to toss arbitrary objects with model-based reinforcement learning," in *2023 9th International Conference on Control, Decision and Information Technologies (CoDIT)*, pp. 1126–1131, 2023.
- [4] K. M. Lynch and M. T. Mason, "Dynamic nonprehensile manipulation: Controllability, planning, and experiments," *The International Journal of Robotics Research*, vol. 18, no. 1, pp. 64–92, 1999.
- [5] M. Bombile and A. Billard, "Bimanual dynamic grabbing and tossing of objects onto a moving target," *Robotics and Autonomous Systems*, vol. 167, p. 104481, 2023.
- [6] A. Zeng, S. Song, J. Lee, A. Rodriguez, and T. Funkhouser, "Tossing-bot: Learning to throw arbitrary objects with residual physics," *IEEE Transactions on Robotics*, vol. 36, no. 4, pp. 1307–1319, 2020.
- [7] M. Monastirsky, O. Azulay, and A. Sintov, "Learning to throw with a handful of samples using decision transformers," *IEEE Robotics and Automation Letters*, vol. 8, no. 2, pp. 576–583, 2023.
- [8] B. Huang, Y. Chen, T. Wang, Y. Qin, Y. Yang, N. Atanasov, and X. Wang, "Dynamic handover: Throw and catch with bimanual hands," in *Conference on Robot Learning (CoRL)*, 2023.
- [9] H. Munn, B. Tidd, P. Böhm, M. Gallagher, and D. Howard, "Whole-body dynamic throwing with legged manipulators," *arXiv preprint arXiv:2410.05681*, 2024.
- [10] Y. Zhang, T. Liang, Z. Chen, Y. Ze, and H. Xu, "Catch it! learning to catch in flight with mobile dexterous hands," in *IEEE International Conference on Robotics and Automation (ICRA)*, pp. 14385–14391, 2025.
- [11] L. Werner, F. Nan, P. Eyschen, F. A. Spinelli, H. Yang, and M. Hutter, "Dynamic throwing with robotic material handling machines," in *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2024.
- [12] Y. Liu, A. Nayak, and A. Billard, "A solution to adaptive mobile manipulator throwing," in *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2022.
- [13] J. Moura, T. Stouraitis, and S. Vijayakumar, "Non-prehensile planar manipulation via trajectory optimization with complementarity constraints," in *IEEE International Conference on Robotics and Automation (ICRA)*, pp. 970–976, 2022.
- [14] E. Coumans and Y. Bai, "Pybullet, a python module for physics simulation for games, robotics and machine learning." <http://pybullet.org>, 2016–2021.